



PRD: AI-Powered Customer Feedback

Document Owner: Navneet Shukla | Last Updated: 3-Jun-2026

Jira Epic	TBD — link epic once created
Delivery Date	TBD
Document Status	In Review
Product Lead	Navneet Shukla
Engineering Lead	TBD
Design Lead	TBD
QA Lead	TBD
Relevant Documents	Existing Streamlit MVP — LLM-powered-customer-feedback-system (GitHub)

Table of Contents

[Table of Contents](#)

[1. Overview](#)

[2. Goals](#)

[3. Success Metrics](#)

[4. Assumptions & Dependencies](#)

[5. Requirements](#)

[6. UX Mockups](#)

[7. Questions](#)

[8. Out of Scope](#)

1. Overview

Background

Businesses today receive customer feedback across multiple channels but lack the infrastructure to respond quickly, consistently, and at scale. Manual responses are slow, inconsistent in tone, and create no feedback loop that tells customers their issues have been addressed.

This project originated as a working Streamlit MVP (Python) that proved the core concept — using a large language model to auto-generate sentiment-aware replies to customer reviews and give admins an analytics view. The MVP is fully functional but built on a single-page Streamlit interface and a flat CSV file, which limits UX quality, scalability, and email communication capabilities.

Why are we doing this?

We are rebuilding and extending the MVP as a production-quality React + Node.js application for three reasons:

- The existing Streamlit UI is not suitable for a modern, app-like customer experience.
- Customers currently receive no post-submission communication — there is no email loop to close the feedback lifecycle.
- Admins have no formal workflow to acknowledge receipt of feedback, commit to a resolution timeline (TAT), or notify customers when an issue is resolved.

How do we know this is a real problem?

The existing MVP has demonstrated the LLM response quality works — the core AI layer is proven. What is unproven is the end-to-end communication loop: customer submits feedback → gets acknowledgement → gets a TAT promise → gets a resolution notification. This PRD closes that gap.

How does this fit into company objectives?

- This product is part of the overall growth strategy — a high-quality feedback loop reduces customer churn and increases trust.
- It provides a reusable internal platform that can be adapted for any product line requiring structured customer communication.
- The AI-generated email responses reduce support staffing cost, contributing indirectly to the annual cost efficiency target.

What problem is this solving?

- Customers submit feedback and hear nothing — no acknowledgement, no timeline, no closure.
- Admins see feedback in a basic list with no structured workflow to commit to a resolution and communicate it back.
- The current flat-file CSV storage and Streamlit UI are not suited to a multi-user, multi-step workflow.

Who are we solving for?

- Customers — end users submitting product or service feedback via a web form.
- Admins — internal team members who review feedback, set TAT commitments, and mark issues as resolved.

What are the user benefits?

- Customers receive an immediate, sentiment-matched AI response in their inbox after submitting feedback.
- Customers receive a TAT acknowledgement email so they know when to expect resolution.
- Customers receive a closure email referencing their original feedback and confirming resolution, encouraging re-engagement.
- Admins get a clean, modern dashboard with clear per-feedback workflow actions — no code or CSV editing required.

2. Goals

What outcome are we trying to achieve? What does success look like?

The following goals define what done looks like for this MVP:

Goal 1 — Modern React UI

Rebuild the customer feedback form and admin dashboard as a React + Node.js application with a clean, modern design that matches the quality of a production SaaS product. The Streamlit UI is retired.

Goal 2 — Sentiment-Aware Email Acknowledgement

Immediately after a customer submits feedback, they receive a transactional email whose tone matches the sentiment of their submission: grateful/warm for positive (4–5 stars), understanding/neutral for neutral (3 stars), apologetic/solution-focused for negative (1–2 stars). Email is sent via a free-tier transactional email provider (Resend.com recommended for MVP).

Goal 3 — Admin TAT Commitment Email

Admins can view each feedback submission and, from the dashboard, send a TAT acknowledgement email to the customer. The email contains the admin-defined turnaround commitment (e.g. '48 hours', '3 business days') and confirms the issue is being reviewed.

Goal 4 — Resolution Closure Email

When an admin marks a feedback item as resolved and clicks 'Send Resolution Email', the customer receives a closure email that: (a) quotes their original feedback, (b) describes what was done to address it, and (c) invites them to try the improvement.

Goal 5 — SQLite Persistence

Replace the CSV flat file with a SQLite database. All feedback records, email events, and resolution notes are persisted reliably. The schema supports the full feedback lifecycle: submitted → acknowledged → in-progress → resolved.

3. Success Metrics

How do we know if we've solved this problem? We will track the following metrics to measure success.

Metric 1 — Email Delivery Rate

≥ 95% of submitted feedback records trigger a successful acknowledgement email delivery (tracked via Resend.com delivery events). Measured in the first 2 weeks post-launch.

Metric 2 — Admin TAT Compliance

≥ 80% of feedback items that receive a TAT commitment email are also marked as resolved within the committed timeframe. Tracked in the admin dashboard.

Metric 3 — Feedback Submission Completion Rate

≥ 90% of users who load the React feedback form complete and submit it. Measured via form-start vs. form-submit events. Validates that the new React UI does not introduce friction compared to the Streamlit version.

Metric 4 — Resolution Email Send Rate

≥ 70% of resolved feedback items have a closure email sent. Tracks whether the admin workflow for resolution communication is being adopted.

Metric 5 — Admin Dashboard Load Time

Admin dashboard renders in < 2 seconds on a standard connection with up to 500 feedback records in the database. Ensures the SQLite + Node.js backend performs adequately at MVP scale.

4. Assumptions & Dependencies

Assumptions

- The Hugging Face Inference API (Qwen2-7B-Instruct) remains available on the free tier for MVP usage volumes. Hardcoded fallback responses cover API downtime.
- Resend.com free tier (100 emails/day, 3,000/month) is sufficient for MVP. No email volume spike is expected during initial testing.
- Admin dashboard access requires no authentication for this MVP — it is accessed on a trusted internal network or via a private URL.
- Customers provide a valid email address at submission time. There is no email verification step in this MVP.

- SQLite is sufficient for MVP data volumes (expected < 10,000 feedback records). Migration to PostgreSQL is a future consideration.
- The Node.js backend and React frontend are deployed together on a single server (e.g. Railway, Render, or local). No microservices separation is needed for this MVP.
- Email tone classification uses the same rating thresholds as the existing MVP: 4–5 stars = positive, 3 stars = neutral, 1–2 stars = negative.

Dependencies

- Hugging Face Inference API — external dependency for LLM response and analysis generation. API key must be provisioned and stored in environment variables.
- Resend.com account — free-tier API key required for transactional email sending. Domain verification recommended but not mandatory for MVP (can use Resend's sandbox domain).
- Node.js runtime (v18+) — backend API server and SQLite integration.
- React (v18+) + Vite — frontend build tooling.
- SQLite via better-sqlite3 — synchronous SQLite driver for Node.js, zero external database service needed.
- The existing feedback_data.csv schema is the source of truth for the initial SQLite migration: id, timestamp, rating, review, ai_response, summary, actions, date.

5. Requirements

All user stories are written for the MVP scope. Priority is Must Have (launch blocker) or Nice to Have (post-launch).

| Acceptance criteria define exactly what an agentic coding tool must implement and verify.

#	Title	User Story	Acceptance Criteria	Priority	Jira	Notes
1	Feedback Submission Form	As a customer, I want to submit a star rating and written review along with my email address so that the business can respond to me personally.	• Form contains: email input (required, validated), star rating selector 1–5 (required), text area for review (min 10 chars, required). • Submit button is disabled until all fields are valid. • On submit, form shows a loading state while the AI response is being generated. • On success, the AI-generated response is displayed inline on the page. • On error (API failure), a fallback response is displayed and the submission is still saved. • Submission is persisted to SQLite with status = 'submitted'.	Must have	TBD	• Email field must use HTML5 email validation. • Min review length: 10 chars.
2	Immediate Acknowledgement Email	As a customer, I want to receive an email immediately after submitting	• Email is sent within 5 seconds of successful form submission. • Email tone matches	Must have	TBD	• Use Resend.com free tier. • Email sender

#	Title	User Story	Acceptance Criteria	Priority	Jira	Notes
		<i>feedback so that I know my feedback was received and feel acknowledged.</i>	sentiment: positive (4–5 stars) = warm/grateful, neutral (3 stars) = understanding, negative (1–2 stars) = apologetic/solution-focused. - Email subject line reflects sentiment (e.g. 'Thank you for your kind words!' / 'We hear you — and we're sorry'). - Email body includes: customer's rating, a snippet of their review, and the AI-generated response text. - If email delivery fails, the error is logged but does not block the form success state.			name: 'Customer Feedback Team'. Sender address: configurable via env var.
3	Admin Feedback Dashboard	<i>As an admin, I want to see all submitted feedback in a dashboard so that I can review customer sentiment and take action.</i>	- Dashboard lists all feedback records sorted by newest first. - Each record shows: timestamp, star rating (visual), email, review text, AI response, current status badge (submitted / acknowledged / in-progress / resolved). - Dashboard displays summary metrics at the top: total reviews, average rating, % positive, % negative. - Rating distribution bar chart and timeline chart are rendered using a charting library. - Refresh button reloads data without a full page reload. - No login required to access the dashboard.	Must have	TBD	- Charts: use Recharts or Chart.js. - Status badge color: blue = submitted, amber = acknowledged, orange = in-progress, green = resolved.
4	AI Analysis per Feedback	<i>As an admin, I want to generate an AI analysis of any feedback item so that I have a structured summary and recommended actions.</i>	- Each feedback card has a 'Generate AI Analysis' button. - On click, the button shows a loading spinner and calls the HF Inference API. - Result is displayed as: one-sentence summary + three numbered action items. - Result is saved to the database (summary and	Must have	TBD	Prompt format must use SUMMARY: / ACTION 1: / ACTION 2: / ACTION 3: markers for reliable parsing.

#	Title	User Story	Acceptance Criteria	Priority	Jira	Notes
			actions fields). • A 'Regenerate' button re-runs the analysis and overwrites the stored result. • If API fails, a hardcoded fallback analysis is displayed and saved.			
5	Admin TAT Commitment Email	<i>As an admin, I want to send a TAT acknowledgement email to a customer so that they know their issue is being reviewed and when to expect a response.</i>	• Each feedback card in the admin dashboard has a 'Send TAT Email' button, visible when status is 'submitted' or 'acknowledged'. • Clicking opens a modal with: a pre-filled TAT message, an editable TAT duration field (e.g. '48 hours', '3 business days'), and a Send button. • On send, the customer receives an email with: their original review, the TAT commitment, and a note that the team is working on it. • The feedback record status updates to 'acknowledged' and the TAT sent timestamp is stored. • The button label changes to 'TAT Sent' and is disabled after sending to prevent duplicate sends.	Must have	TBD	• TAT duration is a free-text field — admin types it manually. • Email subject: 'We're on it — here's what to expect'.
6	Admin Resolution Email	<i>As an admin, I want to mark a feedback item as resolved and send a closure email to the customer so that they know their issue was addressed.</i>	• Each feedback card has a 'Mark Resolved & Send Email' button, visible when status is 'acknowledged' or 'in-progress'. • Clicking opens a modal with: the customer's original feedback (quoted), an editable resolution notes field (what was done), and a Send button. • On send, the customer receives a closure email that: quotes their original review, describes the resolution, and invites	Must have	TBD	• Resolution notes field: required, min 20 chars. • Email subject: 'Your feedback made a difference — here's what changed'.

#	Title	User Story	Acceptance Criteria	Priority	Jira	Notes
			them to try again. - The feedback record status updates to 'resolved' and the resolved timestamp is stored. - Resolution email tone matches original feedback sentiment (positive closure reinforces gratitude; negative closure emphasizes improvement).			
7	SQLite Persistence Layer	<i>As a developer, I want all feedback data and email event history stored in SQLite so that the system has reliable persistence without external database services.</i>	- SQLite database is initialised automatically on server startup if it does not exist. - Schema includes: feedback table (id, timestamp, email, rating, review, ai_response, summary, actions, status, tat_sent_at, resolved_at, resolution_notes), email_events table (id, feedback_id, event_type, sent_at, success). - All API endpoints read/write via SQLite. No CSV files are used. - Database file path is configurable via environment variable (default: ./data/feedback.db). - Basic integrity: feedback id is a UUID, not a timestamp integer.	Must have	TBD	- Use better-sqlite3 for synchronous Node.js SQLite access. - No ORM required — raw SQL is acceptable for MVP.
8	React Modern UI Design	<i>As a customer or admin, I want to use a clean, modern web interface so that the experience feels professional and trustworthy.</i>	- Customer feedback form: card-based layout, smooth form validation states, loading skeleton while AI response generates, success state shows the AI response in a styled card. - Admin dashboard: sidebar navigation, metric cards with trend indicators, responsive table/card list for feedback items, status badges, modal dialogs for TAT and resolution actions.	Must have	TBD	- Use Tailwind CSS for styling. - Component library: shadcn/ui or Radix UI primitives — developer's choice.

#	Title	User Story	Acceptance Criteria	Priority	Jira	Notes
			- Both views are mobile-responsive (min breakpoint: 375px). - No Streamlit dependency — all UI is React components. - Dark/light mode is a nice-to-have, not required for MVP.			
9	Environment Variable Configuration	<i>As a developer, I want all secrets and config values in environment variables so that the app can be deployed safely without hardcoded credentials.</i>	- App reads the following env vars: HF_TOKEN (Hugging Face API key), RESEND_API_KEY, EMAIL_FROM (sender address), DATABASE_PATH (SQLite file path), PORT (default 3001). - App starts with a warning (not a crash) if RESEND_API_KEY is missing — email features degrade gracefully. - App crashes on startup with a clear error message if HF_TOKEN is missing. - A .env.example file is included in the repository with all required variable names and no real values.	Must have	TBD	- Use dotenv for local development. - Never commit .env to version control — add to .gitignore.
10	Sentiment-Aware Email Templates	<i>As a product owner, I want each email type to have three sentiment-specific variants so that the communication always matches the customer's experience.</i>	- Acknowledgement email has three variants: positive template (warm, celebratory tone), neutral template (professional, balanced), negative template (apologetic, action-oriented). - TAT email has three variants matching the same sentiment tiers. - Resolution email has three variants: positive (celebrate the positive relationship), neutral (professional closure), negative (emphasise improvement, rebuild trust). - All email variants are stored as template strings in a dedicated templates module (e.g. src/emails/templates.js).	Must have	TBD	- HTML email is preferred but plain text fallback is sufficient for MVP. - No third-party email template engine required — template literals are fine.

#	Title	User Story	Acceptance Criteria	Priority	Jira	Notes
			Templates support variable interpolation for: customer name/email, star rating, review snippet, AI response, TAT duration, resolution notes.			

6. UX Mockups

Work with the design team to add mockups or wireframes here.

Link the main Figma or Sketch file for reference once designs are created.

The following screens require design coverage before engineering begins:

- Customer Feedback Form — desktop and mobile layouts.
- Feedback Submission Success State — inline AI response card.
- Admin Dashboard — metrics row, feedback list, status badges.
- TAT Commitment Modal — form fields + Send button.
- Resolution Modal — quoted review, resolution notes field, Send button.
- Email Templates — positive, neutral, negative variants for all three email types (acknowledgement, TAT, resolution).

Figma link:

TBD — add link here once designs are created.

7. Questions

QUESTION	OUTCOME
Should the admin be able to edit the AI-generated response before the acknowledgement email is sent, or does it always send the raw LLM output?	TBD — for MVP, send raw LLM output. Editing is a post-MVP feature.
What happens if a customer submits feedback without a valid email address — should the form still accept it?	TBD — current decision: email is required. A future 'guest mode' (no email) can be scoped separately.
Should resolved feedback items be archived or remain visible in the admin dashboard?	TBD — remain visible for MVP with a 'resolved' filter. Archive/hide is post-MVP.
Is Resend.com acceptable as the email provider, or does the team have a preference for SendGrid, Mailgun, or another service?	Resend.com selected for MVP — free tier covers 3,000 emails/month and has a clean Node.js SDK.
Should the TAT duration be a free-text field (admin types '48 hours') or a dropdown (select from pre-set options)?	Free-text for MVP. Dropdown with pre-sets is a nice-to-have for v2.
Does the resolution email need to be admin-approved before sending, or does clicking 'Send' in the modal trigger it immediately?	Immediate send for MVP — no approval workflow required.

8. Out of Scope

The following items are explicitly excluded from this MVP to maintain focus and delivery speed.

The items below are out of scope for this version:

- Admin authentication — login/password or JWT-based auth is not implemented. The dashboard is accessed on a trusted internal network.

- Multi-language support — all UI text and email templates are English only. Translation is a future enhancement.
- Email verification — customers are not asked to verify their email address before submission.
- Admin editing of AI responses — the LLM output is sent as-is in emails. Manual editing before send is post-MVP.
- Dark mode — the React UI is light mode only for this MVP.
- Role-based access control — there is only one admin role; no super-admin or read-only role distinctions.
- PostgreSQL or cloud database — SQLite is the only database for this MVP. Migration is a future task.
- Webhook or CRM integration — no outbound webhooks or CRM sync (e.g. Salesforce, HubSpot) in this MVP.
- Mobile app — the product is a web application only. React Native or other mobile targets are out of scope.
- A/B testing of email templates — all customers receive the same template variant for their sentiment tier.
- Export to PDF or Excel — no report download features in this MVP.
- Pagination or infinite scroll on the admin dashboard — for MVP, all records are loaded (suitable for < 500 records).
- Task 1 (Yelp Rating Prediction) from the original Streamlit app — that feature is not carried over to the React rebuild. It served as an internship assessment artifact only.